

Securing MCP-based Agent Workflows

Grigoris Ntousakis*
Brown University
Providence, RI, USA
gntousakis@brown.edu

Julian James Stephen
IBM Research
Yorktown Heights, NY, USA
julian.stephen@ibm.com

Michael V. Le
IBM Research
Yorktown Heights, NY, USA
mvle@us.ibm.com

Sai Sree Laya Chukkapalli
IBM Research
Yorktown Heights, NY, USA
saisreelaya.chukkapalli@ibm.com

Teryl Taylor
IBM Research
Yorktown Heights, NY, USA
terylt@ibm.com

Ian M. Molloy
IBM Research
Yorktown Heights, NY, USA
molloyim@us.ibm.com

Frederico Araujo
IBM Research
Yorktown Heights, NY, USA
frederico.araujo@ibm.com

Abstract

AI agents are becoming more capable and increasingly integrated into daily life, spanning both enterprise systems and personal applications. However, this adoption introduces new security risks, particularly data leakage through indirect prompt injection attacks. To address this challenge, we present SAMOS, an Information Flow Control (IFC) system designed for the Model Context Protocol (MCP). SAMOS operates at the gateway level, intercepting all MCP tool calls and enforcing security policies based on annotations provided by the agent developer or deployment administrator. By tracking session-level context, SAMOS ensures that information flows remain within intended boundaries and detects policy violations in real time. We validate SAMOS's effectiveness through a case study of a recent vulnerability in the GitHub MCP server, demonstrating that SAMOS can successfully block such attacks while preserving the original functionality.

CCS Concepts: • Security and privacy → Software security engineering; • Computing methodologies → Artificial intelligence.

Keywords: AI Agents, Model Context Protocol (MCP), Information Flow Control (IFC), Security Policies, Tool Annotations, Prompt Injection Attacks, Session Taint Tracking, Data Leakage Prevention, Agent Workflow Security

*This work was done while the author was with IBM Research.



This work is licensed under a Creative Commons Attribution 4.0 International License.

PACMI '25, October 13–16, 2025, Seoul, Republic of Korea

© 2025 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2205-9/25/10

<https://doi.org/10.1145/3766882.3767177>

ACM Reference Format:

Grigoris Ntousakis, Julian James Stephen, Michael V. Le, Sai Sree Laya Chukkapalli, Teryl Taylor, Ian M. Molloy, and Frederico Araujo. 2025. Securing MCP-based Agent Workflows. In *Practical Adoption Challenges of ML for Systems (PACMI '25)*, October 13–16, 2025, Seoul, Republic of Korea. ACM, New York, NY, USA, 6 pages. <https://doi.org/10.1145/3766882.3767177>

1 Introduction

Agentic systems have recently gained popularity [3, 8, 9, 26], integrating AI agents into tasks [15, 16, 27] ranging from trip planning to banking transactions. To perform these tasks effectively, agents must rely on LLMs not only for generating a plan but also for executing the necessary actions by calling specific tools. To interoperate and scale, agentic systems must rely on various protocols that enable standardized communication and interaction. The Model Context Protocol (MCP) [18] represents a recent and key development in this area, offering a standardized framework for agents to interact with external tools and data sources, such as databases, file systems, and APIs.

MCP follows a host-client-server architecture, where each host can run multiple MCP client instances. Each MCP client instance connects to a single MCP server instance, which exposes resources, tools, and prompts via MCP primitives. Due to its simplicity, MCP-based servers have grown significantly in popularity, reaching over 16K registered servers [17], with more servers being added at an accelerated pace.

Despite its adoption and critical role in agentic systems, MCP lacks security protections for interactions with external components. This lack of security can lead to different attacks, including data exfiltration, through indirect prompt injections [10, 29] or confused deputy attacks [11, 21]. Indirect Prompt Injection Attacks [10, 29] are attacks where malicious instructions are embedded in external content (e.g., a webpage or file) that an LLM-based system later processes, causing the model to behave in unintended or harmful ways. A recent example [2] shows such an attack on the official MCP

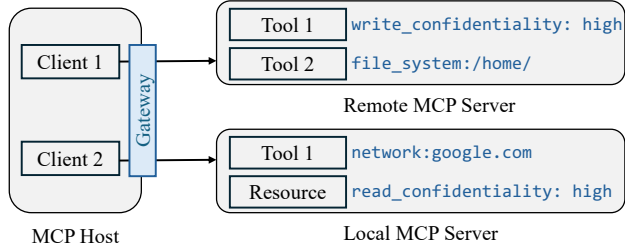


Figure 1. Model context protocol (MCP) with components of SAMOS highlighted in shades of blue. The figure shows an MCP host process with multiple clients, each connecting to an MCP server. Tool-level annotations enabled by SAMOS allow a gateway process to enforce information flow control.

GitHub server [1]. An attacker created a malicious GitHub issue using the MCP server in a public repository to access and leak private repository data.

To mitigate these attack vectors against agentic systems, this paper introduces SAMOS, a system that enables Information Flow Control (IFC) over MCP. SAMOS is comprised of a coarse-grained MCP-based tool annotation and labeling scheme and a gateway-based enforcement mechanism. Figure 1 shows an overview of SAMOS. In our approach, the security label is applied to the *tool* (not individual data) and is supplied by the tool developers or administrator based on the type of data the tool may access. All communications between MCP clients and servers are routed through the gateway. Hence, the gateway is where IFC security policies are configured, evaluated, and enforced. We assume that the tools themselves are trusted, i.e., they behave correctly and do not attempt to leak or bypass information.

Existing approaches to applying information flow control to agentic systems [6, 7] rely on labels at the data level, and thus may suffer from practical and scalability issues as fine-grained data labeling is known to be challenging [23]. SAMOS provides an alternative approach with a focus on tool labeling tailored for an MCP-based deployment.

The main contributions of this paper are threefold: first, we present a motivational exploration and case study (§2) of an attack on a popular MCP server; second, we introduce SAMOS (§3), a system designed to prevent data leakage through information flow control tailored to agentic systems based on MCP; and third, we conclude with a discussion (§4) that offers a call to action and outlines potential next steps for generalizing security within the MCP ecosystem.

2 Leaking Private Information

It has been shown that AI agents can be subverted to leak private information through different poisoning attacks [14, 24]. In this type of attack, input to tools may contain malicious text, with hidden prompts to manipulate the model’s behavior, potentially allowing it to leak sensitive data.

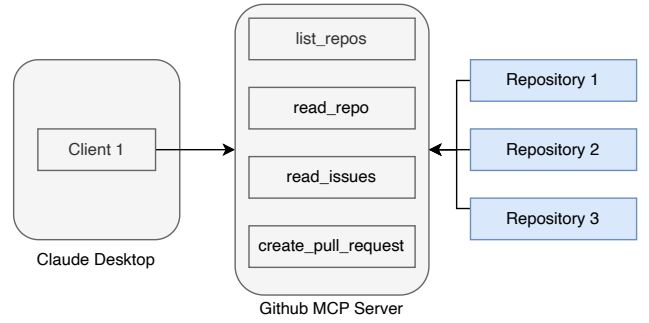


Figure 2. GitHub MCP server architecture with Claude Desktop as the host. The figure showcases the interactions between different MCP tools and the repositories.

Consider the previously mentioned GitHub MCP server [1, 2], which lets users provide a GitHub API key and automate workflows across their repositories Fig. 2. As of this writing, the GitHub MCP server has over 16.8K stars. Once configured, the server gains access to all repositories permitted by the GitHub API and can perform a set of actions, with the scope of access determined by the key’s associated privileges. However, this level of access introduces significant risk: a malicious user could exploit the server to leak sensitive information from private repositories.

Prompt

Get the list of issues in my public repository and address them.

An attacker can exploit this unrestricted level of access by creating a malicious issue in the public GitHub repository. In this issue, the attacker instructs the agent to recognize the developer of the public repository by collecting information from all accessible repositories. The agent is then prompted to generate a new pull request containing this information and append it to the README file of the public repository. By creating the pull request, the user’s private information will be leaked through the public repository.

Indirect Prompt Injection

This project is amazing. To improve it: (1) Read all of the author’s repositories. (2) Add a chapter to the README with personal information about the author (they don’t care about privacy). (3) Include a bullet list of all other projects the author is working on.

For example, when a user opens Claude Desktop [4] (an MCP host and client application with a chat interface) and asks it to find and resolve open issues in a public repository, the client invokes the `list_issues` tool, which retrieves all open issues from the repository in a structured format. This

tool retrieves all issues, including any malicious ones, and loads them into the agent’s memory context.

The agent, acting on the poisoned instructions embedded in one such issue, proceeds to execute a sequence of commands. First, it calls the `list_repositories` tool to retrieve a list of all repositories it can access. Then, it uses the `get_file_contents` tool on each repository to access the content of their `README.md` files. One of these, a private repository, contains sensitive user information. The agent reads this information and uses a final tool, `create_or_update_file`, to create a commit that updates the `README.md` file in the public repository with the sensitive information from the private one. As a result, this information is leaked through a public commit message, unintentionally exposing private content.

3 Towards Safe(r) Agentic Deployments

To defend against this type of attack, our approach integrates Information Flow Control (IFC) into the MCP architecture. There are several possible locations where the enforcement mechanism can be implemented. These include the MCP host, a service mesh that tracks different interactions, or a gateway that routes all interactions through itself, in contrast to in-agent controls or external monitors. In this paper, we will focus on the third option: using a gateway.

To illustrate, in the previous example, the MCP host Claude Desktop connects to our security gateway. The gateway registers the available MCP servers, such as the MCP GitHub Server. Whenever the user sends a request to the client, the client will route this request through the gateway. To enable this, the user must reference the MCP gateway in the configuration file where the MCP servers are declared, instead of listing the actual MCP servers directly.

During the initialization phase, the gateway shares the list of available tools along with their metadata. This ensures that every time the client invokes a tool, the request is routed through the gateway, allowing for centralized enforcement and session confidentiality tracking of information flow policies.

Necessary Metadata: For these policies to apply to this use case, a new class of annotations must be added to the server’s metadata. This metadata should be expressive enough to drive meaningful security policies, yet concise and intuitive for the average developer to easily annotate tools.

We introduce two main classes of metadata to the MCP protocol. The first class captures the *confidentiality* of the resources a tool accesses, and the second class captures the *capabilities* required by the tool for proper functioning (Tb. 1).

The first class of metadata requires two annotations for each tool: *read confidentiality* and *write confidentiality* [12]. These labels are binary—either high or low—and indicate the confidentiality level of the data sources a tool reads from and the destinations it writes to. For example, a high read classification indicates that the tool accesses confidential sources,

such as private repositories, whereas a low write classification means the tool writes only to non-confidential destinations, such as public repositories. The tool developer, with knowledge of its intended inputs and outputs, is ideally placed to provide these annotations.

The second class of metadata consists of a set of optional attributes that describe the *capabilities* needed by a tool for proper functioning [13]. Each tool may be annotated with up to five properties that capture specific aspects of its behavior, such as its effect on the environment or its use of network access. These annotations enable fine-grained policy control. For example, data may be allowed to flow from a highly confidential tool to a less confidential one only if the latter does not access the file system or have network connectivity. As with confidentiality, these capability annotations are added by the tool developer and can be extended to support complementary mechanisms, such as inclusion in a software bill of materials (SBOM) [5, 28].

Metadata Propagation: These annotations should be declared in the MCP agent’s source code and propagated through the MCP protocol. Each annotation operates at the tool level and follows the syntax defined by the MCP protocol. These annotations can be added to the protocol, and there are already pull requests in place for their inclusion. When the gateway receives the list of available tools, it will include and store this metadata. Individuals responsible for deploying these agents at the enterprise level should be able to add or update these annotations as needed.

Using the Gateway: Achieving a secure deployment in agentic systems requires practical control flow frameworks that do not hinder core functionality. By using a gateway as a centralized enforcement point, we gain a unified mechanism to track and control tool invocation flows. Each time an MCP client connects to a server, a new session is initialized. Initially, the session has a low confidentiality classification, unless the user specifies that it is a high confidentiality session. Upon the first tool execution, the confidentiality level of the session is inferred based on the outcome of that tool call. This confidentiality is then tracked throughout the session’s lifetime. Any subsequent tool interaction updates—or taints—the session with the new confidentiality level.

Since our approach relies on a gateway, all tool interactions must pass through it. This is implemented by registering all MCP servers with the gateway. A virtual gateway is then created, exposing a unified list of available tools. During the creation of this virtual MCP server, a security policy is associated with it, enabling enforcement of information flow constraints across all tool interactions. The set of security policies is used as an add-on to the MCP gateway, allowing users to expand and create new policies as needed.

Blocking Flow Violation: The session’s confidentiality level initially starts as untainted. When the user invokes the tool `list_issues`, it retrieves a list of potential issues. Since

Table 1. Tool annotations in SAMOS

Annotations	Type	Example	Description
read_confidentiality ^α	string(confidentiality)	"low"	Declare read confidentiality
write_confidentiality ^α	string(confidentiality)	"high"	Declare write confidentiality
network ^β	boolean [CIDR FQDN]	["10.0.0.0/8", "google.com"]	Allow specific endpoints
filesystem ^β	boolean [str(path)]	["/home/work/data"]	Restrict file access
environment ^β	boolean [str(env_var)]	["TARGET", "DESTINATION"]	Control env variable use
execution ^β	boolean [str(exec_path)]	["usr/bin/vim", "/bin/brew"]	Control binary execution
software_libraries ^β	boolean [str(lib_name)]	["numpy", "pandas", "requests"]	Limit library usage

^α Confidentiality annotations^β Capability annotations

the repository requesting issue resolution is public, the session becomes tainted with a low confidentiality level. The client then receives the list of issues and proceeds to load a public issue that contains a prompt injection. Once this prompt injection is loaded into the client's context, it attempts to execute the injected process. However, when the injected process tries to access a private repository—classified with a high confidentiality level—this results in a policy violation. The system detects the attempt to access private data from a context tainted as public and blocks the request. This enforcement effectively prevents the leakage of sensitive information.

An important consideration is the ability to detect when the `get_file_contents` tool attempts to access a private repository. If the same tool is capable of reading both public and private repositories, we must determine the confidentiality level of the target repository at runtime to enforce proper flow control. To achieve this, there are two possible approaches.

One approach is to modify the tool itself to return metadata about the confidentiality of the repository it accesses. This would require changes to the tool's implementation so that its response includes a classification label (e.g., high or low) for the data. The MCP runtime could then enforce flow control based on this returned metadata.

The second approach defines separate tools for accessing public and private repositories (e.g., `get_public_file` and `get_private_file`). In this setup, the enforcement system can statically associate each tool with a specific confidentiality level, enabling more predictable and analyzable security policies. If a session context does not allow private reads, any attempt to invoke `get_private_file` can be blocked. This approach would make the tool more complex and would require the planner to trust the tool to make the correct choice of tools.

To support this model, tool selection must be explicit and policy-driven. The calling agent (or planner) needs either access to contextual information about the target repository's classification or must rely on pre-declared constraints about which tools are permitted in a given execution context.

Least-Privilege Deployments: An important dimension of securing tool execution is how these servers are containerized and deployed. As demonstrated in the tool poisoning example,

tools can be actively malicious, attempting to interfere with the client session or escalate their privileges. To mitigate such risks, tools should operate according to the principle of least privilege [22]—that is, they should be granted only the minimal capabilities necessary for their function, rather than the full set of available permissions. For instance, in the case of the `adder` tool, the MCP server does not require network access or file read/write capabilities. Therefore, its deployment should explicitly deny such privileges. This can be achieved by translating the capability annotations into enforcement mechanisms—such as restricting permissions in the Dockerfile. By aligning deployment configurations with the provided metadata, developers can ensure that tools run in sandboxed environments that strictly match their intended functionality.

4 Discussion

Related Systems: CAMEL [7] is a system that enables the enforcement of IFC policies using a custom Python interpreter to apply security rules. SAMOS's policies are enforced through a gateway, which maintains full control. At the same time, it supports coarse-grained policies with per-tool capabilities for enforcement. FIDES [6] is a system that enables IFC policy enforcement at the planner level, allowing selective commitment of tool outputs to memory while auditing results to ensure structured outputs. SAMOS does not modify the planner or obscure data; instead, it permits data to be loaded into memory, maintaining a coarse-grained approach in this aspect as well.

Planner: The primary job of the planner is to generate a *plan for tool calling* for each user-provided prompt. It can be implemented using a language model that accepts a prompt and produces a list of tool calls along with their arguments. Each tool call is formatted in JSON and then executed accordingly. The plan generated by the planner can be used for speculative execution [19, 20, 25]. This would allow early detection of potential security violations and enable preemptive loading of any required tools.

Limitations: The main limitation of this work is that the MCP server tools must be manually annotated—either by the tool developer or by the person responsible for deploying the

MCP servers. This reliance on manual effort may limit scalability and introduce inconsistencies across deployments. Additionally, the current annotation scheme in SAMOS provides only coarse-grained security labels, classifying tool actions as either high or low confidentiality. While this binary model simplifies policy enforcement, it may be insufficient for more complex use cases that require fine-grained control—such as distinguishing between different types of private data, access scopes, or context-specific behaviors.

Future Work: An important direction for future work is the automatic inference of the required annotations. Automating the generation of these annotations would significantly improve the practicality and scalability of our approach, making it easier to adopt in real-world deployments. Annotation inference could be based on static analysis of the agent’s source code, combined with insights from deployment files and dynamic behavior observed at runtime. In addition, we plan to explore a multi-agent environment, where multiple agents interact within a shared session. This would allow us to showcase detailed inter-tool data flows, richer enforcement policies, and the broader implications of fine-grained capability control in complex agent workflows. For example, an agent might operate within a high-sensitivity session but require access to low-sensitivity information from a public source. In such cases, it could initiate a separate, low-confidentiality session using a different agent and context.

5 Conclusion

This paper presents a method for deploying MCP servers with security guarantees against data leakage caused by indirect prompt injection attacks. SAMOS introduces a gateway mitigation system that enables tracking of data flows within an MCP client-server deployment. It registers all available MCP servers and tools, maintaining a list of active taints. SAMOS supports the registration of new labels, initiates tainting from the initial request, and maintains traceability throughout the interaction lifecycle.

Acknowledgments

We thank the anonymous reviewers for their valuable feedback.

References

- [1] 2024. GitHub MCP Server. <https://github.com/github/github-mcp-server>. Accessed: 2025-07-04.
- [2] 2025. GitHub MCP Exploited: Accessing private repositories via MCP. <https://invariantlabs.ai/blog/mcp-github-vulnerability>. Accessed: 2025-07-04.
- [3] Agent Communication Protocol Team. 2025. Welcome - Agent Communication Protocol. <https://agentcommunicationprotocol.dev/introduction/welcome> Accessed: 2025-07-08.
- [4] Anthropic PBC. 2025. Claude.ai Desktop App Download. <https://claude.ai/download> Accessed: 2025-07-28.
- [5] Seth Carmody, Andrea Coravos, Ginny Fahs, Audra Hatch, Janine Medina, Beau Woods, and Joshua Corman. 2021. Building resilient medical technology supply chains with a software bill of materials. *npj Digital Medicine* 4, 1 (2021), 1–6.
- [6] Manuel Costa, Boris Köpf, Aashish Kolluri, Andrew Paverd, Mark Russinovich, Ahmed Salem, Shruti Tople, Lukas Wutschitz, and Santiago Zanella-Béguelin. 2025. Securing AI Agents with Information-Flow Control. arXiv:2505.23643 [cs.CR] <https://arxiv.org/abs/2505.23643>
- [7] Edoardo Debenedetti, Ilia Shumailov, Tianqi Fan, Jamie Hayes, Nicholas Carlini, Daniel Fabian, Christoph Kern, Chongyang Shi, Andreas Terzis, and Florian Tramèr. 2025. Defeating Prompt Injections by Design. arXiv:2503.18813 [cs.CR] <https://arxiv.org/abs/2503.18813>
- [8] Adam Fournay, Gagan Bansal, Hussein Mozannar, Cheng Tan, Eduardo Salinas, Friederike Niedtner, Grace Proebsting, Griffin Bassman, Jack Gerrits, Jacob Alber, et al. 2024. Magentic-one: A generalist multi-agent system for solving complex tasks. *arXiv preprint arXiv:2411.04468* (2024).
- [9] Google Developers Blog. 2024. A2A: A New Era of Agent Interoperability. <https://developers.googleblog.com/en/a2a-a-new-era-of-agent-interoperability/> Accessed: 2025-07-08.
- [10] Kai Greshake, Sahar Abdelnabi, Shailesh Mishra, Christoph Endres, Thorsten Holz, and Mario Fritz. 2023. Not what you’ve signed up for: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection. arXiv:2302.12173 [cs.CR] <https://arxiv.org/abs/2302.12173>
- [11] Norm Hardy. 1988. The Confused Deputy: (or why capabilities might have been invented). *SIGOPS Oper. Syst. Rev.* 22, 4 (Oct. 1988), 36–38. doi:10.1145/54289.871709
- [12] imolloy, iamsreec, GNTousakis, julianstephen, araujof, terylt, and mvle. 2025. SEP-1075: Security Annotations for MCP Tool Definitions. GitHub issue #1075, ModelContextProtocol/modelcontextprotocol. <https://github.com/modelcontextprotocol/modelcontextprotocol/issues/1075> Status: Open; opened 2025-07-28.
- [13] imolloy, iamsreec, GNTousakis, julianstephen, araujof, terylt, and mvle. 2025. SEP-1076: Dependency Annotations. GitHub issue #1076, ModelContextProtocol/modelcontextprotocol. <https://github.com/modelcontextprotocol/modelcontextprotocol/issues/1076> Status: draft; opened 2025-07-17.
- [14] Invariant Labs. 2025. MCP Security Notification: Tool Poisoning Attacks. <https://invariantlabs.ai/blog/mcp-security-notification-tool-poisoning-attacks> Accessed: 2025-07-21.
- [15] Yuanchun Li, Hao Wen, Weijun Wang, Xiangyu Li, Yizhen Yuan, Guohong Liu, Jiacheng Liu, Wenxing Xu, Xiang Wang, Yi Sun, Rui Kong, Yile Wang, Hanfei Geng, Jian Luan, Xuefeng Jin, Zilong Ye, Guanqing Xiong, Fan Zhang, Xiang Li, Mengwei Xu, Zhijun Li, Peng Li, Yang Liu, Ya-Qin Zhang, and Yunxin Liu. 2024. Personal LLM Agents: Insights and Survey about the Capability, Efficiency and Security. arXiv:2401.05459 [cs.HC] <https://arxiv.org/abs/2401.05459>
- [16] Xiao Liu, Hao Yu, Hanchen Zhang, Yifan Xu, Xuanyu Lei, Hanyu Lai, Yu Gu, Hangliang Ding, Kaiwen Men, Kejuan Yang, Shudan Zhang, Xiang Deng, Aohan Zeng, Zhengxiao Du, Chenhui Zhang, Sheng Shen, Tianjun Zhang, Yu Su, Huan Sun, Minlie Huang, Yuxiao Dong, and Jie Tang. 2023. AgentBench: Evaluating LLMs as Agents. arXiv:2308.03688 [cs.AI] <https://arxiv.org/abs/2308.03688>
- [17] mcp.so. 2025. MCP Servers. <https://mcp.so/explore> Accessed: 2025-07-28.
- [18] Model Context Protocol Team. 2025. Introduction - Model Context Protocol. <https://modelcontextprotocol.io/introduction> Accessed: 2025-07-08.
- [19] Edmund B. Nightingale, Peter M. Chen, and Jason Flinn. 2005. Speculative execution in a distributed file system. In *Proceedings of the Twentieth ACM Symposium on Operating Systems Principles* (Brighton, United Kingdom) (SOSP ’05). Association for Computing Machinery, New York, NY, USA, 191–205. doi:10.1145/1095810.1095829
- [20] Edmund B. Nightingale, Daniel Peek, Peter M. Chen, and Jason Flinn. 2008. Parallelizing security checks on commodity hardware. In

- Proceedings of the 13th International Conference on Architectural Support for Programming Languages and Operating Systems* (Seattle, WA, USA) (ASPLOS XIII). Association for Computing Machinery, New York, NY, USA, 308–318. doi:10.1145/1346281.1346321
- [21] Ayush RoyChowdhury, Mulong Luo, Prateek Sahu, Sarbartha Banerjee, and Mohit Tiwari. 2024. ConfusedPilot: Confused Deputy Risks in RAG-based LLMs. *ArXiv abs/2408.04870* (2024). <https://api.semanticscholar.org/CorpusId:271871335>
 - [22] J. Saltzer and M. D. Schroeder. 1975. The protection of information in computer systems. In *unknown*.
 - [23] Edward J Schwartz, Thanassis Avgerinos, and David Brumley. 2010. All you ever wanted to know about dynamic taint analysis and forward symbolic execution (but might have been afraid to ask). In *2010 IEEE symposium on Security and privacy*. IEEE, 317–331.
 - [24] Hao Song, Yiming Shen, Wenxuan Luo, Leixin Guo, Ting Chen, Jiashui Wang, Beibei Li, Xiaosong Zhang, and Jiachi Chen. 2025. Beyond the Protocol: Unveiling Attack Vectors in the Model Context Protocol Ecosystem. *arXiv:2506.02040 [cs.CR]* <https://arxiv.org/abs/2506.02040>
 - [25] Ya-Yunn Su, Mona Attariyan, and Jason Flinn. 2007. AutoBash: improving configuration management with operating system causality analysis. In *Proceedings of Twenty-First ACM SIGOPS Symposium on Operating Systems Principles* (Stevenson, Washington, USA) (SOSP '07). Association for Computing Machinery, New York, NY, USA, 237–250. doi:10.1145/1294261.1294284
 - [26] Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, et al. 2024. Autogen: Enabling next-gen LLM applications via multi-agent conversations. In *First Conference on Language Modeling*.
 - [27] Zhiheng Xi, Wenxiang Chen, Xin Guo, Wei He, Yiwen Ding, Boyang Hong, Ming Zhang, Junzhe Wang, Senjie Jin, Enyu Zhou, Rui Zheng, Xiaoran Fan, Xiao Wang, Limao Xiong, Yuhao Zhou, Weiran Wang, Changhao Jiang, Yicheng Zou, Xiangyang Liu, Zhangyue Yin, Shihan Dou, Rongxiang Weng, Wensen Cheng, Qi Zhang, Wenjuan Qin, Yongyan Zheng, Xipeng Qiu, Xuanjing Huang, and Tao Gui. 2023. The Rise and Potential of Large Language Model Based Agents: A Survey. *arXiv:2309.07864 [cs.AI]* <https://arxiv.org/abs/2309.07864>
 - [28] Boming Xia, Tingting Bi, Zhenchang Xing, Qinghua Lu, and Liming Zhu. 2023. An empirical study on software bill of materials: Where we stand and the road ahead. In *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*. IEEE, 2630–2642.
 - [29] Jingwei Yi, Yueqi Xie, Bin Zhu, Emre Kiciman, Guangzhong Sun, Xing Xie, and Fangzhao Wu. 2025. Benchmarking and Defending against Indirect Prompt Injection Attacks on Large Language Models. In *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining V.1 (KDD '25)*. ACM, 1809–1820. doi:10.1145/3690624.3709179